

PROJECT ΔΟΜΩΝ ΔΕΔΟΜΕΝΩΝ 2021

26/06/2021

Συνεργάτες:

1) Βέργος Γεώργιος AM:1072604

2) Τσούλος Βασίλειος AM:1072605

3) Γκίκας Πέτρος AM:1072512

4) Βλάχος Σταύρος AM:1072489

Emails:

1)up1072604@upnet.gr

2)up1072605@upnet.gr

3)up1072512@upnet.gr

4)up1072489@upnet.gr

Μέρος 1^ο (Searching and Sorting algorithms)

1)

Με τη βοήθεια δομής Stock με πεδία Open,High,Low,Close,OpenInt,Volume και Date(τη σπάμε σε 3 ακέραιους) φτιάχνουμε από το αρχείο πίνακα τέτοιων δομών με πλήθος στοιχείων όσες οι γραμμές του αρχείου με τους αριθμούς που χειριζόμαστε. Για τους αλγορίθμους Mergesort και Quicksort (ταξινομούμε ως προς το πεδίο Open) παρατηρούμε ότι ο mergesort έχει καλύτερο άνω φράγμα στη χειρότερη περίπτωση (πολυπλοκότητα $O(n \log n)$) σε σχέση με τον quicksort που είναι $O(n^2)$. Έτσι κατά την εκτέλεση του προγράμματος ο mergesort χρειάζεται λιγότερο χρόνο για να ταξινομήσει το πίνακα. Ο mergesort δουλεύει καλά ανεξαρτήτως μεγέθους του πίνακα ενώ ο quicksort είναι καλός για πίνακες μικρού μεγέθους. Έτσι στη περίπτωση μας ο mergesort είναι πιο αποδοτικός. Ωστόσο ο mergesort έχει μεγαλύτερες απαιτήσεις χώρου στη μνήμη από τον quicksort. Ο Mergesort εκτελεί λιγότερες στοιχειώδεις πράξεις από τον quicksort.

2)

Με τη βοήθεια δομής Stock με πεδία Open,High,Low,Close,OpenInt,Volume και Date(τη σπάμε σε 3 ακέραιους) φτιάχνουμε από το αρχείο πίνακα τέτοιων δομών με πλήθος στοιχείων όσες οι γραμμές του αρχείου με τους αριθμούς που χειριζόμαστε. Για τους αλγορίθμους HeapSort και Counting Sort (ταξινομούμε ως προς το πεδίο Close) παρατηρούμε ότι ο Counting sort έχει πολυπλοκότητα χειρίστης περίπτωσης $O(n+k)$ όπου n το πλήθος των στοιχείων του πίνακα και k η μέγιστη τιμή από αυτές που ταξινομούμε. Για τον Heapsort η πολυπλοκότητα χειρίστης περίπτωσης έχουμε $O(n \log n)$. Ο counting sort έχει το μειονέκτημα ότι λειτουργεί μόνο σε θετικούς ακέραιους γι' αυτό και κάνουμε στρογγυλοποίηση στην πλησιέστερο ακέραιο μιάς και οι περισσότερες τιμές του του είναι floats.

3)

Για τον υπολογισμό της interpolation Search απαιτείται το data set να ακολουθεί ομοιόμορφη κατανομή και μόνο σε αυτή την περίπτωση η απόδοση είναι καλύτερη από τον Binary Search. Επίσης σε μικρά Data sets η binary Search αποδεικνύεται συνήθως καλύτερη αφού η interpolation Search απαιτεί περισσότερους υπολογισμούς. Η πολυπλοκότητα του interpolation search μέσης περίπτωσης είναι $O(\log \log n)$ σε ομοιόμορφη κατανομή. Στη χειρότερη περίπτωση έχουμε πολυπλοκότητα $O(n)$. Ο binary search έχει πολυπλοκότητες καλύτερης ,μέσης και χειρότερης $O(1)$, $O(\log n)$, $O(n)$ αντίστοιχα.

4)

Ο αλγόριθμος δυική αναζήτηση παρεμβολής έχει χρονική πολυπλοκότητα μέσης περίπτωσης $O(\log \log n)$ και χειρότερο χρόνο $O(\sqrt{n})$. Με τη παραλλαγή του BIS όπου αντικαθιστούμε στον κώδικα τη γραμμή : $i++$ με $i=2*i$ βελτιώνουμε το χρόνο χειρότερης περίπτωσης σε $O(\log n)$ χωρίς να αλλάξει ο χρόνος μέσης περίπτωσης.

ΜΕΡΟΣ 2^ο BST'S AND HASHING

A)

Δουλεύουμε με κομβοπροσανατολισμένο AVL δέντρο(η πληροφορία είναι στους κόμβους και όχι στα φύλλα) . Μέσω της συνάρτησης *eisagogi* εισάγουμε κόμβους στο δέντρο από το αρχείο η οποία χρησιμοποιεί συναρτήσεις όπως η *Ypsozygisi* ,*max_of_subtrees* ,*Ypsos* ,*aristeri_peristrofi*,*deksia_peristrofi* προκειμένου οι κόμβοι του δέντρου να έχουν συντελεστή ισορροπίας είτε -1,0,+1 ώστε το δέντρο να είναι ισοσταθμισμένο. Έπειτα σύμφωνα με τα ζητούμενα έχουμε συναρτήσεις όπως η *troporoihsh* όπου διατρέχουμε το δέντρο με τον ακόλουθο τρόπο: εάν η ημερομηνία που δώσαμε είναι μικρότερη από αυτή του τρέχοντα κόμβου πηγαίνει αριστερά αλλιώς δεξιά. Ομοίως και με την αναζήτηση του όγκου βάσει ημερομηνίας που δίνουμε. Για την διαγραφή κόμβου βάσει ημερομηνίας έχουμε τη συνάρτηση *diagrafi* όπου κάνουμε τις κατάλληλες περιστροφές στους κόμβους που έχουν συντελεστή ισορροπίας διαφορετικό από -1,0,+1, ώστε το avl δέντρο να είναι ισοζυγισμένο. Το δέντρο διατάσσεται ως προς το Date.

B)

Το avl δέντρο διατάσσεται ως προς το πεδίο Volume . Εφόσον το avl δέντρο είναι ένα δυαδικό δέντρο αναζήτησης τότε για κάθε κόμβο θα ισχύει δεδομένα αριστερού παιδιού<δεδομένα πατέρα<δεδομένα δεξιού παιδιού. Έτσι η ελάχιστη και η μέγιστη τιμή του όγκου(εφόσον το δέντρο διατάσσεται ως προς αυτόν) θα βρίσκονται στο αριστερότερο και δεξιότερο φύλλο αντίστοιχα. Το υλοποιούμε με τις συναρτήσεις *min_Volume*,*max_Volume*.

Γ)

Χρησιμοποιούμε τη συνάρτηση *ascii_values* για να προσθέσουμε τις ascii τιμές των χαρακτήρων των ημερομηνιών από το αρχείο που είναι string της μορφής : "YYYY-MM-DD" προκειμένου να πάρουμε το κλειδί (key) της ημερομηνίας αυτής. Έπειτα για την εισαγωγή της στον πίνακα χρησιμοποιούμε τη συνάρτηση κατακερματισμού $h(k)=k \bmod m$ όπου m =γραμμές του αρχείου (όσοι δηλαδή και οι κόμβοι). Υλοποιούμε το hashing με αλυσίδες μέσω διασυνδεδεμένης λίστας(χρήση δεικτών) έτσι ώστε να αποφύγουμε τις συγκρούσεις(collisions). Όταν η θέση του πίνακα μίας ημερομηνίας με συγκεκριμένο κλειδί είναι ίδια με αυτήν ενός ήδη αποθηκευμένου κόμβου τότε έχουμε σύγκρουση και απλά μέσω αλγορίθμου χειρισμού της λίστας προσθέτουμε το στοιχείο στο τέλος της. Για την αναζήτηση του όγκου βάσει ημερομηνίας

που δίνεται αρχικά υπολογίζεται το κλειδί της και η θέση του πίνακα στην οποία αντιστοιχεί. Στη συνέχεια διατρέχουμε τη λίστα γραμμικά μέχρι το τέλος της για να το βρούμε. Όμοια λειτουργεί και η συνάρτηση `trororoihs` για να αλλάξουμε την τιμή του `Volume`. Η διαγραφή συγκεκριμένης ημερομηνίας πραγματοποιείται με σύνδεση του δείκτη `next` του προηγούμενου κόμβου από αυτόν που θέλουμε να διαγράψουμε με τον επόμενο κόμβο αυτού που θέλουμε να διαβάσουμε .